

FAI Final Project: UCT-based Texas Hold’Em Agent

Jhih-Jhong Lin
B11902015

1 Introduction

Texas Hold’em, a variant of Poker, has long been a complex poker game as viewed by game theory. Various methods and strategies have been proposed to enhance AI agents’ competitiveness in the game, such as reinforcement learning [4], supervised learning [5], and even pure probability-based method [6]. In this project, we devise and implement a Texas hold’em agent based on the UCT (*Upper Confidence Bound for Trees*) algorithm that determines the best action to take on each hand, with an eye to outperforming the pure-probability-based method proposed by Wu [6], which serves as not only our baseline approach, but also one of the optimization techniques applied to our agent.

§2 defines our setting for the game Texas hold’em used in this project. Then, we briefly introduce our design, as well as important implementation-dependent building blocks, of our approach in §3. In §4 we list all optimization techniques we have applied to our approach, which are meant to enhance the action-selection policy to outmatch the baseline agents (*baseline1* to *baseline7*) in grading, and surpass the effectiveness of the baseline approach: probability-based agent [6]. The evaluation results demonstrating the efficacy of variants of our agent, each tuned differently, are in §5, followed by the discussion and our conclusion in §6.

In brief, the method we have submitted is our agent based on smooth UCT with dynamic constant selection, integrated with probability-based agent modelling players’ strategy, and with parameters optimized.

2 Game Setting

In this work, we focus on two-player no-limit Texas hold’em (2-NLHE), in which our agent competes with a baseline agent (namely, *baseline1* to *baseline7*) head-to-head with no universal upper bound on a player’s raise, apart from the number of chips in their stack. A *match* is defined to consist of 5 *games*, and a *game* is defined to consist of at most 20 *rounds*, each includes 5 streets. At the start of a game, each player’s stack is initialized to contain 1000 chips, while players keep their chips to the next round after one ends. A player is considered winner if and only if their stack contains more chips than the other’s. These definitions align with the competition rules used in grading.

We further define the **win rate** of a player as the fraction of their winning games out of 100 independent games, in which the first-round dealer is selected uniformly at random and the two players alternate as dealer in subsequent rounds.

3 A Basic UCT-based Agent

3.1 Algorithm

Our agent mainly leverages the UCT algorithm, the most popular algorithm in the Monte-Carlo tree search (MCTS) family [2], to search for the best action to take given the current state. On each hand, the agent models the current state as the root node of the

state (sub)tree, on which each child of a state is the state to which the corresponding valid action (e.g. fold, call \$100, raise \$1000) will lead the state.

Since the UCT algorithm has already been taught in class, we do not elaborate the entire process here. Briefly speaking, the algorithm is nothing more than MCTS with a function UCB1 as a score function. The algorithm firstly (1) iteratively selects the “best” child from the root down to a node with an unexplored child. Then, starting from its child, it (2) begins a Monte Carlo simulation, i.e., it “simulates” a random playout by repeatedly picking a child to go down at random until reaching a leaf, i.e., a terminal state. By “best” we mean having a larger UCB1 value. After (3) calculating the “reward” for the terminal state, it (4) propagates the reward all the way back to the root and goes back to step (1). By numerous times of random simulations, the resulting “rewards” calculated for each child of the root state (i.e. the current state) will eventually approximate the expected score at the terminal state for each action taken. Last the not least, the abovementioned UCB1 function, which defines the priorities of the states from which the algorithm simulates the game, is defined for each non-root state s as

$$UCB1(s) \triangleq Q(s) + c \sqrt{\frac{\ln N(p_s)}{N(s)}}$$

where $Q(s)$ is the estimated reward for the player (or rather, the agent) if playing from state s , $N(s)$ is the number of visits to state s , and p_s denotes the parent state of s [3]. The estimated reward $Q(s)$ is computed from the reward achieved in previous simulations starting from any descendant of s . In addition, the constant c , which balances the resulting sum between the *exploitation* term $Q(s)$ on the left and the *exploration* term $\sqrt{\frac{\ln N(p_s)}{N(s)}}$ on the right, is typically $\sqrt{2}$. In this project, we follow the exact UCT algorithm described by Browne et al. [2].

In the rollout stage (step (3)), right after determining the state to start the simulation from, our agent will have all unknown hole cards randomly assigned, and then it will start randomly picking a valid action for each acting player until the simulated round ends (that is, the showdown), as described above. Moreover, once the simulation comes to an end, our agent computes the reward as the difference between its stack size at the root state and its stack size at the current state. That is, we define the estimated reward $Q(s)$ at state s as

$$Q(s) \triangleq \frac{1}{N(s)} \sum_{k=1}^{N(s)} (\text{Stack}(T(s, k)) - \text{Stack}(s))$$

where $\text{Stack}(s)$ denotes the size of the agent’s stack size at the s , and $T(s, k)$ denotes the terminal state (the state after showdown) in the k -th rollout that starts from a descendent of s .

3.2 Rollout Strategy

It is worth mentioning that in rollout stage (i.e. Monte Carlo simulation step), we simply model our opponent’s strategy to be “always calling”. That is, we have every simulated player choose to “call” until the simulated round ends. This avoids our agent’s burden to perform heavy computations just to predict our opponent’s action in the simulated states, enabling fast rollouts and thereby increasing the number of simulations. Despite these merits, always calling or even always going all in, however, are apparently naive strategies, which our opponent is highly unlikely to adopt. Consequently, our simulated playouts might not reflect real outcomes, resulting in inaccurate estimates of the true expected rewards. To mitigate the issue, we have instead employed probability-based action-selection strategy for rollouts, which we will later describe in §4.

3.3 Choice of the Exploration Parameter

The constant c , usually called the *exploration parameter*, in UCB1 formula plays an important role in controlling when the exploitation term or the exploration term dominates the total UCB1 value. Even though in most applications of UCT taking $c \triangleq \sqrt{2}$ suffices, stack sizes, and thus their differences, are typically of the order of magnitude of at least 2. As a result, exploitation term $Q(s)$ is often greatly outweighs the exploration term, stopping the UCT algorithm from selecting an underexplored state for simulation if c is not big enough. Hence, we must find a suitable value for c such that c is able to strike a balance between the two terms.

4 Optimizations

We have applied several techniques to our method, seeking an enhancement of its intelligence. However, these techniques turn out to be ineffective in our evaluation.

4.1 Smooth UCT

Smooth UCT [3], as its name suggests, is a variant of the UCT algorithm. It mixes between the max-UCB1 strategy (as in UCT) and the average strategy, which, as claimed by the authors, “converges to a Nash equilibrium in certain classes of games.” In particular, on the selection stage (step (1) in §3), smooth UCT selects the “average strategy” with a probability $1 - \eta_k$, and resorts to the original max-UCB1 method (which is introduced in §3.1) with probability η_k . η_k an iteration k -adapted sequence with $\eta_k \rightarrow \gamma > 0$ as $k \rightarrow \infty$ [3]. For the ease of implementation, we use the same construction of η_k as in the paper, i.e., we let η_k be

$$\eta_k \triangleq \max \left(\gamma, \eta \left(1 + d\sqrt{N(s)} \right)^{-1} \right)$$

where η, γ, d are constants (i.e. parameters). Besides, at a state s the “average strategy” is defined to randomly select a child c , each with probability $\frac{N(c)}{N(s)}$. This strategy for child selection is hence based on randomness, as opposed to the max-UCB1 strategy whose outcome is deterministic.

4.2 Probability-Based Rollouts

Wu [6] proposed from a mathematical view a pure probability-based method to select the best action for an agent, yielding a rationally-bahaved agent and even outperforming the RL-based method in his work. To address the false modeling issue elaborated in §3.2, we employ this method to predict players’ selections of actions in rollouts. However, still, we suspect that this method fail to capture real-world player behavior accurately enough.

In brief, this method models the distribution of the net profit in each round as a normal distribution $\mathcal{N}(\mu, \sigma^2)$ [6]. Based the formulation, the *game rate*—the probability of winning a game—for a player who has earned t chips (i.e. holding $1000 + t$ chips in stack) with C rounds remaining can be easily computed by

$$G(t, C) = 1 - (-t; C\mu, C\sigma^2)$$

where the notations are borrowed from his work. Since the game rate is now a function of the number of chips t the player has before the round starts and actually computable, at each hand the agent simply enumerates all valid actions, computes the (expected) game rate after taking each action, and select the action that gives the highest game rate. Note also that the parameters required in this approach are μ and σ .

4.3 Dynamic Exploration Parameter

To address the problem where the magnitude of the exploitation term, $Q(s)$, is not on par with that of the exploration term, besides fine-tuning the fixed exploration parameter c in the UCB1 calculation, we also employ a method described by Heinrich et al. [3] that dynamically updates c so that it always balances between the two terms. In this method, the exploration parameter, which is not fixed anymore, is defined to be

$$c \triangleq \min(c_{cap}, \text{Potsize} + k \cdot \text{Remaining Betting Potential})$$

where c_{cap}, k are the parameters and the *remaining betting potential* is referred to as “the maximum possible amount that players can add to the pot in the remainder of the episode” [3].

4.4 Parameter Tuning

Since our design and the applied optimization techniques involve plenty of parameters, inclusive of c (if fixed), $\eta, \gamma, d, \mu, \sigma, c_{cap}$, and k , manually tuning these values surely is a lot of hassle. Therefore, we make use of the Optuna framework [1] to test our agent’s performance on different parameter values against the given baseline agents, and then set the parameters to values that are likely to yield the best performance, i.e., the highest score.

5 Evaluation

To evaluate the competitiveness of our basic UCT-based agent described in §3, as well as the 3 optimization techniques we adopt in §4, in 2-NLHE, we measure the *win rate* of the following variants of our agent, which are integrated with different combinations of optimizations, against the given baseline agents, with the probability-based agent described in §4.2 serving as the baseline:

- (1) *basic*: Our basic UCT-based agent without optimization
- (2) *dynC*: Basic agent with dynamic exploration parameter
- (3) *smoothUCT*: Basic agent with smooth UCT

Table 1: Win rates of various agents against each baseline over 100 2-NLHE games.

Agent \ Baseline	baseline0	baseline1	baseline2	baseline3	baseline4	baseline5	baseline6	baseline7
basic	0.51	0.58	0.69	0.77	0.50	0.37	0.42	0.48
dynC	0.47	0.54	0.61	0.84	0.49	0.30	0.40	0.36
smoothUCT	0.54	0.61	0.69	0.85	0.39	0.45	0.44	0.38
dynC+smoothUCT	0.54	0.68	0.66	0.81	0.42	0.22	0.44	0.45
PBagent	0.52	0.68	0.71	0.84	0.35	0.44	0.49	0.41
dynC+PBagent	0.48	0.67	0.62	0.75	0.46	0.39	0.40	0.38
smoothUCT+PBagent	0.63	0.64	0.68	0.70	0.37	0.42	0.46	0.36
dynC+smoothUCT+PBagent	0.53	0.56	0.65	0.61	0.41	0.46	0.48	0.36
PBagent w/o MCTS	0.32	0.29	0.28	0.61	0.11	0.16	0.14	0.27

- (4) *dynC+smoothUCT*: Basic agent with dynamic exploration parameter and smooth UCT
- (5) *PBagent*: Basic agent with probability-based rollout
- (6) *dynC+PBagent*: Basic agent with dynamic exploration parameter and probability-based rollout
- (7) *smoothUCT+PBagent*: Basic agent smooth UCT and probability-based rollout
- (8) *dynC+smoothUCT+PBagent*: Our agent with all optimizations applied
- (9) *PBagent w/o MCTS*: The pure probability-based agent described in the work [6]

Table 1 shows our experimental results.

5.1 Comparison

As can be seen in Table 1, the basic UCT-based agent (*basic*) already achieves win rates above 40% against most baselines, indicating that pure UCT already provides a solid foundation.

On the other hand, the pure probability-based baseline (*PBagent w/o MCTS*) achieves an average win rate of only 0.2725, far below even the unoptimized UCT agent (*basic*), which reaches a 0.54 average. This gap confirms that some form of MCTS is essential: without MCTS, the agent wins less than 28% of games, whereas every UCT-based variant more than doubles that performance.

Adding dynamic exploration parameter (*dynC*) actually drops our average win rate to around 50.1%, and smooth UCT (*dynC+smoothUCT*) only raises it slightly to 54.4%. Using probability-based action-selection strategy on rollouts inside UCT (*PBagent*) gives the biggest bump to 55.5%, but that increase is small and varies a lot from one opponent to another. Mixing two or all three tweaks always makes the agent perform worse than plain UCT.

To sum up, aside from some slight gains on win rates caused by the random nature of 2-NLHE, none of these tweaks gives a clear, consistent improvement. As the tiny or negative changes in win rate aren't worth the extra complexity, these optimizations are showned to be ineffective in our setting.

6 Conclusion

In this project, we have shown that a basic UCT-based agent substantially outperforms a pure probability-based strategy, confirming the importance of MCTS in 2-NLHE. Our three proposed enhancements—dynamic exploration parameter, smooth UCT and probability-based rollouts—failed to yield consistent or significant improvements over

the basic, unmodified UCT-based agent. Although rollouts guided by probability yielded a modest gain in isolation, combining any two or all three techniques degraded performance. These results suggest that added algorithmic complexity is not justified by the marginal benefits observed. Future work may explore alternative rollout strategies in MCTS or a deeper integration between search and learned value estimates.

References

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (Anchorage, AK, USA) (KDD '19). Association for Computing Machinery, New York, NY, USA, 2623–2631. <https://doi.org/10.1145/3292500.3330701>
- [2] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. 2012. A Survey of Monte Carlo Tree Search Methods. *IEEE Transactions on Computational Intelligence and AI in Games* 4, 1 (2012), 1–43. <https://doi.org/10.1109/TCIAIG.2012.2186810>
- [3] Johannes Heinrich and David Silver. 2015. Smooth UCT search in computer poker. In *Proceedings of the 24th International Conference on Artificial Intelligence* (Buenos Aires, Argentina) (IJCAI'15). AAAI Press, 554–560.
- [4] Alex Kashi, Vedang Lad, and Håkon Grini. [n. d.]. *Building an Efficient Poker Agent Using RL*. <https://www.vedanglad.com/projects/1.pdf>
- [5] Luis Filipe Teófilo and Luis Paulo Reis. 2011. Building a No Limit Texas Hold'em Poker Agent Based on Game Logs Using Supervised Learning. In *Autonomous and Intelligent Systems*, Mohamed Kamel, Fakhri Karay, Wail Gueaieb, and Alaa Khamis (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 73–82.
- [6] Yen-Siang Wu. [n. d.]. *Probability-Based Agent for Texas Hold'em Poker*. <https://www.csie.ntu.edu.tw/~b09902097/HoldemAgent.pdf>